Assignment 3. Maze Escape

Course	Design and Analysis of Algorithms
Topics	Disjoint Set Union, minimum spanning trees (Kruskal, Prim), DFS and BFS, strongly connected components, topological ordering, shortest paths in DAGs
Weight	100 points: assignment 50, defense 50
Language	Java 17 or newer. No other language and no graph or DSU libraries.
Submission	Link to your GitHub repository in Moodle. A late submission is not graded (syllabus).

1. The task

A maze is a graph: a cell is a vertex, a wall between two neighbouring cells is an edge. You build a maze with Kruskal and your own DSU, find the way out with DFS, and rebuild the maze as the cheapest possible one with Kruskal and Prim. Then you find the one extra wall that shortens the way out the most.

The maze also has one-way slides between cells. You find their zones (strongly connected components), order the zones topologically and find the cheapest route from the start to the exit.

Every student has personal files made from the student ID, so every number in your output is your own.

2. Your input files

The starter project contains tools/MakeVariant.java. Run it with your student ID:

```
java tools/MakeVariant.java YOUR_ID data
```

It creates two files in the folder data.

walls.txt. The first line is "R C W": rows, columns, number of inner walls. Each of the next W lines is "a b w": a wall between the neighbouring cells a and b, and w, the cost of breaking it. All costs are different. Lines of walls are numbered from 1.

slides.txt. The first line is "n S": the number of cells and the number of slides. Each of the next S lines is "a b w": a one-way slide from cell a to cell b that takes w seconds. Slides ignore walls.

Cells are numbered from 0, row by row: the cell in row r and column c has the number r*C + c. The start is cell 0 (top left), the exit is cell R*C - 1 (bottom right). A tiny example, 2 rows and 2 columns:

```
walls.txt     slides.txt     cells
2 2 4         4 4            0 1
0 1 17        0 1 3          2 3
0 2 5         1 0 2
1 3 9         1 2 4
2 3 23        2 3 5
```

3. Program and output

```
javac -d out src/main/java/*.java
java -cp out Main data/walls.txt data/slides.txt
```

The program prints the 15 lines below, each as "name: whole number", in this order. Other lines are allowed. The program must not wait for keyboard input.

Output line	Meaning	Mission
Broken	Number of walls broken when you take the walls in the order of the file (Mission 1).	1
Kept	Number of walls that stay.	1
Last broken wall	Line number in walls.txt of the last wall that was broken.	1
MST cost	Total cost of the walls broken in the cheapest maze.	2
Heaviest broken wall	The largest cost among the walls broken in the cheapest maze.	2
Path length	Number of moves from the start to the exit in the maze of Mission 1.	3
DFS visited	Number of cells DFS has marked at the moment it marks the exit; the start and the exit are counted. DFS tries the neighbours in the order up, right, down, left.	3

Dead ends	Number of cells with exactly one open door (the start and the exit count too).	3
Zones	Number of strongly connected components of the slides. A zone is a set of cells such that from any cell of it you can reach any other one by slides. A cell without a way back is a zone of its own.	4
Largest zone	The number of cells in the biggest zone.	4
Is DAG	1 if the slides have a topological order (no cycle), otherwise 0.	4
Flow stages	The graph of zones has one vertex per zone and an edge X to Y if some slide goes from a cell of X to a cell of Y. Flow stages is the number of zones on its longest chain.	5
Fastest route cost	In the graph of zones the edge X to Y costs the time of the cheapest slide from X to Y; walking inside a zone is free. The cost of the cheapest route from the zone of the start to the zone of the exit.	5
Best shortcut gain	In the maze of Mission 1 break one more wall so that the way out is as short as possible. The gain is the old Path length minus the new one. 0 if no wall helps.	6
Shortcut wall	Line number of that wall; the smallest line number if several walls are equally good; -1 if no wall helps.	6

Output for the tiny files above, to check your reading of the rules:

```
Broken: 3   Kept: 1   Last broken wall: 3   MST cost: 31   Heaviest broken wall: 17
Path length: 2   DFS visited: 3   Dead ends: 2
Zones: 3   Largest zone: 2   Is DAG: 0   Flow stages: 3   Fastest route cost: 9
Best shortcut gain: 0   Shortcut wall: -1
```

4. Missions (50 points)

Mission	What to do	Points
0. DSU	Class DisjointSet with find, union (returns false if the cells are already in one group), connected and count. Union by size and path compression. JUnit tests.	5
1. Build the maze	Take the walls in the order of the file. Break a wall if union returns true; otherwise the wall stays. README: why Broken is always R*C - 1, and why Kept is always (R-1)*(C-1).	7
2. The cheapest maze	Break walls so that all cells are still connected and the total cost is the smallest. Implement Kruskal with your DSU and Prim with a priority queue; both must give the same MST cost.	8
3. Escape	DFS from the start to the exit in the maze of Mission 1. Think about a maze of 1000 by 1000 cells: the program must not overflow the stack. README: why there is exactly one path.	6
4. Zones	Find the zones of the slides with Kosaraju. Try to build a topological order of all the cells; the program must say clearly when it is impossible (Is DAG). README: why the second pass of Kosaraju goes on the reversed graph.	8
5. Fastest route	Build the graph of zones, order it topologically, and in that order compute Flow stages and the cheapest route to the exit. Do not use a priority queue here. README: why this order is enough, and the cost of the pass.	6
6. Shortcut	Find the best single extra wall in the maze of Mission 1 without a new search for every wall: BFS from the start, BFS from the exit, then check each standing wall in constant time. README: a table comparing this with the simple way (one BFS per wall) on a maze of 300 by 300 cells.	5
7. Picture, timing, README	The program writes maze.txt (the line numbers of the broken walls of Mission 1, one per line) and path.txt (the cells of the way out, one per line). Open them with walls.txt in tools/maze_viewer.html and save the picture as maze.png (1 point). Time of Kruskal and of Prim on mazes of 100x100, 300x300 and 1000x1000 cells: 5 runs each, System.nanoTime, one plot (2 points). README: how to run, and a table with the time and space complexity of every algorithm you wrote (2 points).	5

Stages: missions 0 to 2 (week 6), missions 3 to 5 (week 7), missions 6 and 7 at the end. The dates are in Moodle.

5. Submission

Submit the link to your GitHub repository in Moodle, for example https://github.com/username/maze-escape. The repository must be public.

- All source code is in the repository. Use a proper workflow: a branch for each stage (stage-1, stage-2, stage-3), merged into main with a pull request, and commit messages that say what was done. The main branch always compiles.
- The report is the README.md of the repository: your ID, the output of your program, the answers, the tables and the plot. Use the template from the starter project.
- data/walls.txt, data/slides.txt and maze.png are in the repository.

A submission without a working link or without the README report is not accepted.

6. Defense (50 points)

The defense takes 5 minutes and is worth 50 points: one mini task and two questions. You come alone, with your laptop and your code open. Phones, messages and AI tools are not allowed.

7. Rules

- Your own work only. Copied code or a shared solution means zero for the assignment and a report to the disciplinary committee, as in the syllabus.
- You must be able to explain your own code. If you cannot, the points for the assignment can be reduced.
- Do not use ready-made graph or DSU libraries. The Java standard library (collections, priority queue) is allowed.